



Sandya Mannarswamy

CODE SPORT

This month's column continues the discussion on software bloat, and also covers software abstraction and abstraction penalty.

Welcome to a great new year! Last month, we had featured a bunch of programming questions. Since I received the correct solutions to only a few of them, I would like to keep them open for this month as well. We will discuss the solutions in next month's column. This month, let's continue our discussion on software bloat, by looking at how software abstraction, a cornerstone of modern programming paradigms, contributes inadvertently to software inefficiency.

Software abstraction

Abstraction is the holy grail of programming in high-level languages. Abstraction allows programmers to focus their attention on what needs to be done, instead of how it needs to be done. The programmers can employ abstractions without concern about how they are implemented. Consider a simple example of a library 'sort' routine that programmers use. The programmers need not be concerned with how the sort routine is implemented; all they need to know is, what are the arguments that need to be supplied to the sort routine, and how does it return the results. That allows them to use the 'sort' routine with minimal effort. Similarly, consider basic data types such as the integer or float, supported by a high-level programming language. Programmers can use these data types and the operations supported on them without having to be concerned with how those operations are implemented, or how the integer data type is internally represented inside the machine. Abstraction is useful because it allows programmers to think and spend their time on solving specific problems, while making use of the abstractions provided by the high-level programming language. It improves programmer productivity and facilitates ease of programming.

Many high-level programming languages support abstract data types such as sets, lists or maps, in addition to built-in types such as the integer, character or float.

These abstract data types are either directly supported by the language, as in functional languages like LISP, or supported through the standard library, as in the case of C++/Java. For instance, C++ STL provides multiple abstract data types, such as set, list, map, queue or stack. Similarly, Java provides the Collections framework, which includes set, list and map. An Abstract Data Type (or ADT as it is popularly known) provides an interface for a certain set of operations on particular kinds of data, shielding users from having to know how the data is represented, and how the operations are implemented. This enables an assembly-line kind of approach, wherein these ADTs can be assembled and used by the programmer to build complex software. For instance, programmers who use the C++ STL container 'queue' need not know how the 'queue' is implemented. They only need to know what operations are supported by 'queue', so that they can use it in their code.

In general terms, what users can do with ADTs can be classified into two kinds of operations: 'queries' and 'updates'. Queries are operations that compute results based on the data present in the ADT. They typically do not modify the ADT. On the other hand, updates are operations that update the ADT. The following table gives some of the operations supported on the 'list' ADT in C++ STL:

<code>reference front()</code>	<code>void insert(iterator pos, size_type n, const T& x)</code>
<code>const_reference front() const</code>	<code>iterator erase(iterator pos)</code>
<code>reference back()</code>	<code>iterator erase(iterator first, iterator last)</code>
<code>const_reference back() const</code>	<code>void clear()</code>
<code>void push_front(const T&)</code>	<code>void resize(n, t = T())</code>
<code>void push_back(const T&)</code>	<code>void splice(iterator pos, list& l)</code>